

EXPERIMENT 1: 8 QUEEN PROBLEM

Aim:

To develop a Python program that solves the 8-Queens Problem using the Backtracking Algorithm by placing eight queens on an 8×8 chessboard so that no two queens attack each other.

Objective:

- To understand the concept of the Backtracking algorithm.
- To place 8 queens on an 8×8 chessboard safely.
- To ensure that no two queens share the same row, column, or diagonal.
- To display one valid arrangement of the queens.

Algorithm:

Step 1: Start.

Step 2: Create an empty 8×8 chessboard.

Step 3: Place the first queen in the first column.

Step 4: Check whether the position is safe by verifying:

- Left side of the row
- Upper-left diagonal
- Lower-left diagonal

Step 5: If the position is safe, place the queen.

Step 6: Move to the next column and repeat the process.

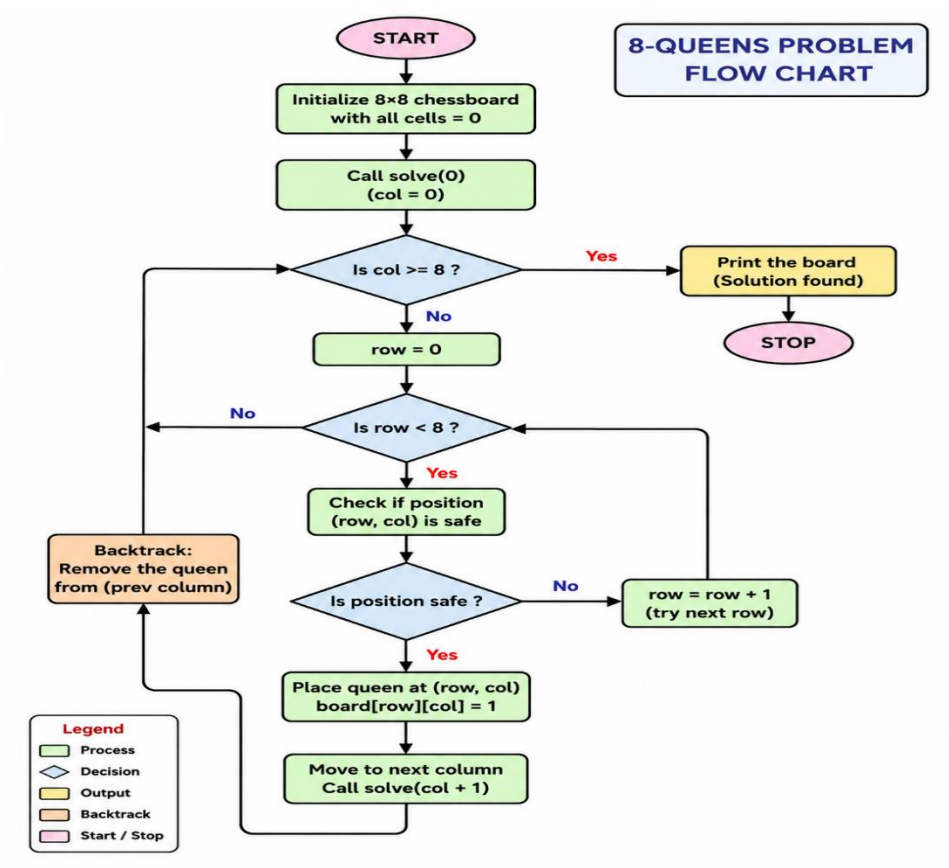
Step 7: If no safe position is available, remove the previously placed queen (Backtracking) and try another row.

Step 8: Continue until all 8 queens are placed.

Step 9: Display the chessboard solution.

Step 10: Stop.

Flow chart :



Source code :

N = 8

```
board = [[0 for _ in range(N)] for _ in range(N)]
```

```
def is_safe(row, col):
```

```
    # Check left side
```

```
    for i in range(col):
```

```
        if board[row][i] == 1:
```

```
            return False
```

```
    # Check upper-left diagonal
```

```
    i, j = row, col
```

```
while i >= 0 and j >= 0:
```

```
    if board[i][j] == 1:
```

```
        return False
```

```
    i -= 1
```

```
    j -= 1
```

```
# Check lower-left diagonal
```

```
i, j = row, col
```

```
while i < N and j >= 0:
```

```
    if board[i][j] == 1:
```

```
        return False
```

```
    i += 1
```

```
    j -= 1
```

```
return True
```

```
def solve(col):
```

```
    if col >= N:
```

```
        return True
```

```
for row in range(N):
```

```
    if is_safe(row, col):
```

```
        board[row][col] = 1
```

```
        if solve(col + 1):
```

```
            return True
```

```
board[row][col] = 0
```

```
return False
```

```
if solve(0):
```

```
    print("8-Queens Solution:\n")
```

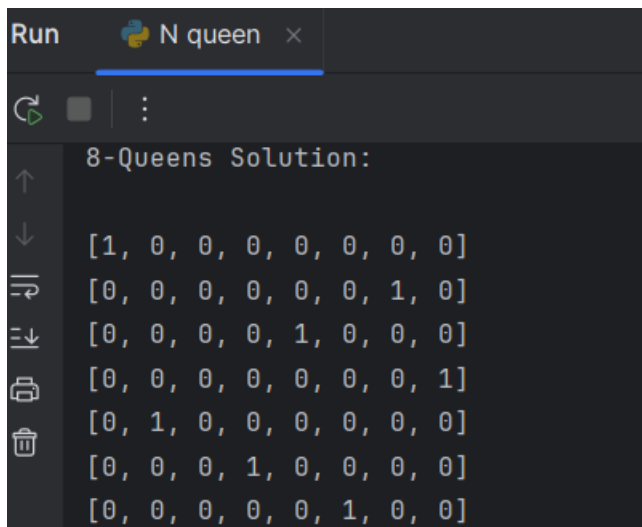
```
    for row in board:
```

```
        print(row)
```

```
else:
```

```
    print("No solution exists.")
```

Output :

A screenshot of a Python IDE window titled "N queen". The output console shows the text "8-Queens Solution:" followed by eight rows of a board representation. Each row is a list of eight integers, where a '1' indicates a queen and '0' indicates an empty space. The queens are located at (row, col) positions: (0,0), (1,7), (2,5), (3,8), (4,1), (5,3), (6,6), and (7,4).

```
8-Queens Solution:  
[1, 0, 0, 0, 0, 0, 0, 0]  
[0, 0, 0, 0, 0, 0, 1, 0]  
[0, 0, 0, 0, 1, 0, 0, 0]  
[0, 0, 0, 0, 0, 0, 0, 1]  
[0, 1, 0, 0, 0, 0, 0, 0]  
[0, 0, 0, 1, 0, 0, 0, 0]  
[0, 0, 0, 0, 0, 1, 0, 0]
```

Result :

The Python program successfully solves the 8-Queens Problem using the Backtracking Algorithm and displays a valid arrangement of eight queens on the chessboard such that no two queens attack each other.

